

PCC-203-COM : Operating Systems

(2024 Pattern) (Semester - III)

Unit 1	Introduction to Operating System	Marks
Basics of Operating Systems: Objectives & Functions, Evolution of OS, Types of Operating Systems, OS Service, System Calls: Introduction, Types of System Calls OS structure: Layered Approach, Monolithic, Microkernel Operating Systems Introduction to Linux OS: Components of Linux system, Basic Shell commands		14

The Idea: An Operating System (OS) is the most important piece of software on a computer. It's the **master controller** that acts as a bridge between you (the user) and the complex, raw computer hardware. Without an OS, a computer is just a useless box of electronics.

Part 1: The Role and Evolution of Operating Systems**1.1 Objectives & Functions of an Operating System**

An OS has two primary objectives:

- 1. Convenience (To act as an Extended Machine / Virtual Machine):**
 - **Goal:** To hide the complex, messy details of the hardware and provide the user with a simple, clean, and easy-to-use interface.
 - **How:** It creates abstractions. You don't need to know how a hard disk works; you just see "files" and "folders." You don't need to manage CPU cycles; you just run "programs." The OS makes the computer convenient to use.
- 2. Efficiency (To act as a Resource Manager):**
 - **Goal:** To manage all the computer's resources (CPU, memory, disk drives, printers, etc.) efficiently, fairly, and securely.
 - **How:** It acts like a traffic cop, deciding who gets to use which resource and for how long. It resolves conflicts between different programs and users, ensuring the system runs smoothly and doesn't waste resources.

Key Functions of an OS:

- **Process Management:** Managing the life cycle of programs (creating, scheduling, and terminating them).
- **Memory Management:** Allocating and deallocating memory space to programs as they need it, and ensuring they don't interfere with each other.
- **File Management:** Organizing data into files and directories, controlling access, and managing storage space.
- **I/O Device Management:** Handling communication with all input/output devices (keyboard, mouse, monitor, printer).
- **Security and Protection:** Protecting the system from unauthorized access and ensuring that programs can only access the resources they are allowed to.
- **Networking:** Managing network connections and communication protocols.

1.2 Evolution of Operating Systems

The history of operating systems is a story of increasing complexity and automation.

1. Serial Processing (1940s-1950s): No OS.

- Programmers interacted directly with the hardware using plug boards and switches. Incredibly inefficient, with lots of wasted time between jobs.

2. Batch Systems (1950s-1960s): The First "OS".

- Jobs with similar needs were bundled together in a "batch" and run one after the other. A simple program called a "resident monitor" would automatically load and run the next job, improving efficiency.
- **No user interaction.**

3. Multiprogramming (1960s-1970s): The Juggling Act.

- The big breakthrough. The OS could keep **several jobs in memory at once**. When one job had to wait for a slow I/O operation (like reading from a tape), the OS would switch the CPU to another job.
- **Goal:** Maximize CPU utilization. This was the birth of CPU scheduling.

4. Time-Sharing (Multitasking) Systems (1970s onwards): Interactive Computing.

- An extension of multiprogramming. The OS switches the CPU between jobs so frequently (a few milliseconds per job) that it creates the illusion that multiple users are interacting with their programs simultaneously.

- This made interactive computing (where the user gets an immediate response) possible. This is the foundation of modern desktop and server OS.

5. Personal Computer Systems (1980s onwards):

- Focused on user convenience and responsiveness (GUIs, mouse).
Examples: MS-DOS, Windows, macOS.

6. Distributed & Networked Systems:

- Multiple computers connected by a network, working together but appearing to the user as a single, powerful system.

7. Mobile Systems:

- OS designed for smartphones and tablets, focusing on power efficiency, touch interfaces, and connectivity. Examples: Android, iOS.

1.3 Types of Operating Systems

1. Batch Operating System:

- **The Idea:** This is one of the earliest forms of an OS, designed for maximum efficiency when computers were massive, expensive machines that could only do one thing at a time. The goal was to eliminate the wasted "setup time" between jobs.
- **How it Works:**
 1. **Job Submission:** Programmers would prepare their jobs (the program code, data, and control instructions) on physical media like punch cards.
 2. **Batching:** An operator would collect these jobs and group them into batches with similar needs (e.g., all programs written in FORTRAN, all programs needing a specific tape drive).
 3. **Execution:** The OS (at this stage, a simple program called a "resident monitor") would load the entire batch into the computer. It would then run the first job to completion, then automatically load and run the second job, and so on, until the entire batch was finished.
- **Key Characteristics:**
 - **No User Interaction:** Once a job is submitted, the user cannot interact with it. They just have to wait for the output, which might be hours later.
 - **Maximizes Machine Time:** The primary goal is to keep the expensive CPU busy by reducing idle time between jobs.
 - **Lack of Responsiveness:** Not suitable for any task that requires user input or immediate results.

- **Example Use Case:** Payroll processing, generating monthly bank statements, or large-scale scientific calculations where a massive dataset is processed without human intervention.

2. Multiprogramming Operating System:

- **The Idea:** This was the next major evolution. The OS developers realized that the CPU was often idle even in a batch system because it was waiting for slow I/O devices (like a tape drive or a printer) to finish. Multiprogramming keeps the CPU busy by giving it something else to do during these waiting periods.
- **How it Works:**
 1. **Memory Management:** The OS keeps **several jobs in memory at the same time**.
 2. **Job Scheduling:** The OS picks one job to run.
 3. **Context Switching:** When that job needs to perform a slow I/O operation, instead of letting the CPU sit idle, the OS **suspends** that job and immediately **switches** the CPU to another job that is in memory and ready to run. When the I/O operation for the first job is complete, it is put back in the ready queue.
- **Key Characteristics:**
 - **Multiple Programs in Memory:** Requires sophisticated memory management.
 - **CPU Scheduling:** Requires an algorithm to decide which of the ready jobs to run next.
 - **Goal: Maximize CPU Utilization.** The main purpose is to ensure the CPU is always working on *something*.
- **Example Use Case:** This is the foundation of almost all modern operating systems. Any system that can run a word processor while downloading a file in the background is using multiprogramming.

3. Multitasking (Time-Sharing) Operating System:

- **The Idea:** Time-sharing is a logical extension of multiprogramming that is designed for **interactive use**. It switches between jobs so frequently and so quickly that it creates the illusion that multiple users or programs are running simultaneously.

- **How it Works:**
 1. The OS allocates a very short period of CPU time, called a **time slice** or **quantum** (e.g., 10-20 milliseconds), to each job.
 2. When a job's time slice expires, the OS forcibly suspends it (even if it's not waiting for I/O) and switches the CPU to the next job in the queue.
 3. This context switching happens so fast that each user feels like they have dedicated access to the computer.
- **Key Characteristics:**
 - **Focus on Responsiveness:** The primary goal is to provide a short response time to each user.
 - **Frequent Context Switching:** Requires a highly efficient scheduler.
 - **Foundation of Modern OS:** All modern desktop operating systems like Windows, macOS, and Linux are time-sharing systems.
- **Example Use Case:** You are typing in a word processor, listening to music, and have a web browser open. The OS is rapidly switching the CPU's attention between all these applications, giving each one a small slice of time.

4. Real-Time Operating System (RTOS):

- **The Idea:** An RTOS is a specialized OS where the correctness of a computation depends not only on the logical result but also on the **time at which the result is produced**. It's all about predictability and meeting deadlines.
- **Types:**
 - **Hard Real-Time OS:**
 - **Guarantee:** Provides an absolute, iron-clad guarantee that a critical task will *always* meet its deadline. Missing a deadline is considered a total system failure.
 - **Use Cases:** Life-critical systems like flight control systems in an airplane, anti-lock braking systems in a car, pacemakers, and industrial robotics.
 - **Soft Real-Time OS:**
 - **Guarantee:** Prioritizes critical tasks over all other tasks and tries its best to meet deadlines, but it does not provide an absolute guarantee. Missing a deadline degrades performance but doesn't cause a total system failure.
 - **Use Cases:** Systems where fast response is important but not life-or-death, such as live audio/video streaming (a dropped frame is bad, but not catastrophic) or online financial transaction systems.

5. Distributed Operating System:

- **The Big Idea:** A distributed OS manages a collection of separate, networked computers, but it makes them appear to the user as if they are a **single, unified system**. The complexity of the underlying network and multiple machines is completely hidden.
- **Key Characteristics:**
 - **Transparency:** Hides the fact that resources (files, CPUs) are distributed across multiple machines.
 - **Resource Sharing:** A user on one machine can easily access resources on another machine without even knowing it.
 - **High Scalability and Reliability:** If one computer in the system fails, the OS can often continue running by shifting its workload to other machines.
- **Example Use Case:** High-performance computing clusters, large-scale cloud platforms like Google Cloud or Amazon AWS.

6. Network Operating System (NOS):

- **The Idea:** A Network OS is an OS that is designed to run on a central server in a network. Its primary job is **not** to create a single-system illusion, but to provide **shared services** to a group of independent client computers.
- **How it Works:**
 - The server runs the NOS (e.g., Windows Server, a version of Linux).
 - The client computers run their own desktop OS (e.g., Windows 11, macOS).
 - The NOS provides functionalities like:
 - **File and Print Sharing:** A central place to store shared files and manage print jobs.
 - **User and Security Management:** A central place to manage user accounts, passwords, and permissions.
 - **Directory Services:** Like a company-wide address book (e.g., Active Directory).

- **Key Difference from Distributed OS:** In a NOS, the users are **aware** that there are multiple, separate computers. They know they are connecting to a "server." In a Distributed OS, this distinction is hidden.

Part 2: OS Services and System Calls

2.1 OS Services

These are the high-level services that the OS provides to users and programs to make their lives easier. They are the "features" of the OS.

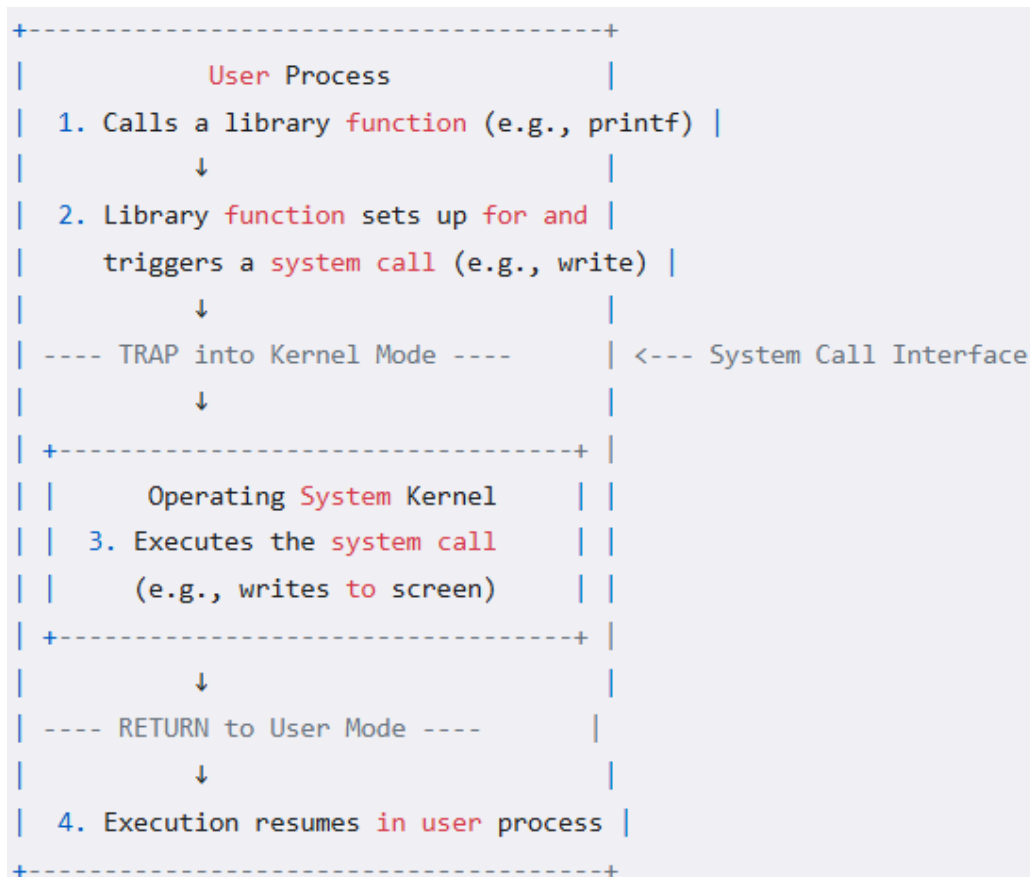
- **User Interface (UI):** How the user interacts with the OS. Can be a **Command-Line Interface (CLI)**, a **Graphical User Interface (GUI)**, or a **Batch Interface**.
- **Program Execution:** Loading a program into memory, running it, and managing its execution.
- **I/O Operations:** Providing a simple way for programs to read from and write to files and devices, hiding the complex hardware details.
- **File System Manipulation:** Creating, deleting, reading, writing, and managing permissions for files and directories.
- **Communication:** Enabling processes to exchange information, either on the same computer (**Inter-Process Communication - IPC**) or across a network.
- **Error Detection:** Constantly checking for errors in the CPU, memory, I/O devices, or user programs, and taking appropriate action.
- **Resource Allocation:** Distributing system resources among multiple users or jobs.
- **Accounting:** Keeping track of which users use how much and what kinds of computer resources.
- **Protection and Security:** Ensuring that access to system resources is controlled and that the system is safe from external threats.

2.2 System Calls: The Gateway to the OS

- **Introduction:** A **system call** is the programmatic way in which a user program requests a service from the operating system's kernel. It's the **only entry point** for a user program to access privileged hardware or kernel services.
- **Why are they needed? (Dual-Mode Operation):**

- Modern CPUs operate in at least two modes: **User Mode** and **Kernel Mode** (also called supervisor mode).
- User Mode:** The mode in which user applications run. This mode has restricted access to hardware.
- Kernel Mode:** The mode in which the OS kernel runs. This mode has privileged, unrestricted access to all hardware.
- A system call is a special, controlled mechanism that allows a program to transition from User Mode to Kernel Mode, ask the OS to perform a privileged task on its behalf, and then return to User Mode. This protects the system from faulty or malicious user programs.

• Textual Diagram of the Process:



Types of System Calls:

- Process Control:** `fork()` (create a process), `exit()` (terminate), `wait()` (wait for a child).

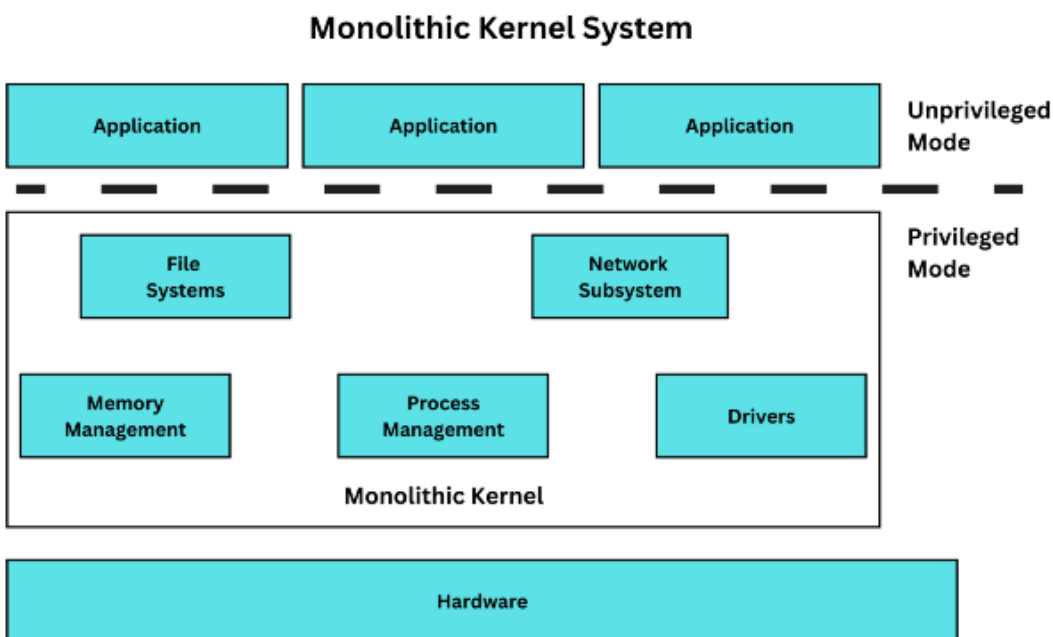
- **File Management:** open(), read(), write(), close().
- **Device Management:** ioctl() (control a device), read(), write().
- **Information Maintenance:** getpid() (get process ID), gettimeofday() (get current time).
- **Communication:** pipe() (create a channel), shmget() (get shared memory), socket() (create a network socket).

Part 3: OS Structure - How is the OS Itself Built?

There are different philosophies for how to structure the internal components of an operating system kernel.

1. Monolithic Structure

- **The Idea:** The "big ball of mud" approach. The entire operating system is a single, large program running in kernel mode. All services (process scheduling, memory management, file systems, device drivers) are compiled into one large binary.
- **How it Works:** All components are tightly coupled and can call each other directly.
- **Textual Diagram :**



Pros : Excellent performance : Communication between components is just a simple function call, which is very fast.

- **Cons: Hard to maintain and debug.** A bug in one part (e.g., a device driver) can bring down the entire system. It's difficult to add new features without potentially breaking something else.
- **Examples:** Traditional UNIX, Linux (though Linux uses a modular monolithic design), MS-DOS.

2. Layered Approach

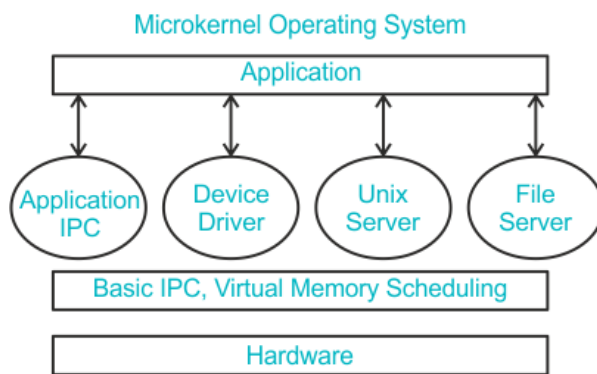
- **The Idea:** The OS is broken down into a number of layers, each built on top of the one below it. The bottom layer is the hardware, and the highest layer is the user interface.
- **How it Works:** Each layer can only use functions and services provided by the layer directly beneath it. This enforces a strict structure.
- **Textual Diagram:**

```
Layer 5: User Interface
-----
Layer 4: User Programs
-----
Layer 3: I/O Management
-----
Layer 2: Memory Management
-----
Layer 1: CPU Scheduling
-----
Layer 0: Hardware
```

- **Pros: Good for design and debugging.** Because of the strict layering, you can debug one layer at a time. It simplifies construction.
- **Cons: Poor performance.** A request from a high layer might have to pass through many intermediate layers to be serviced, adding significant overhead to each call.

3. Microkernel Operating Systems

- **The Idea:** The "minimalist" approach. The kernel itself should be as small as absolutely possible, providing only the most essential services.
- **How it Works:** Only the absolute bare minimum—like basic process communication (IPC) and memory management—runs in the privileged kernel mode. All other services (file systems, device drivers, networking) run as separate **user-mode processes** called "servers."



- **Pros: High security and reliability.** If a device driver or file server crashes, it's just a user-mode process that crashes; it won't bring down the entire OS. It's also very flexible and easy to extend.
- **Cons: Performance overhead.** Communication between the user-mode servers must go through the microkernel's message passing (IPC), which is much slower than a simple function call in a monolithic kernel.
- **Examples:** Mach, QNX (used in cars and industrial systems), Minix.

Part 4: Introduction to the Linux OS

1. Components of the Linux System

Linux is not just a kernel; it's a complete operating system composed of several parts:

- **The Kernel:** The core of the OS. It manages the CPU, memory, and peripherals. It is the master controller.

- **The Shell:** The **command-line interface (CLI)**. It's a program that takes commands typed by the user and tells the kernel to execute them. It's the primary way to interact with the system at a low level.
- **System Libraries:** A collection of pre-written functions that applications use to interact with the kernel (e.g., the C library, libc). These libraries wrap the raw system calls in easy-to-use functions like printf().
- **System Utilities:** The vast collection of command-line tools that provide the basic functionality of the OS, such as ls, cp, mv, grep, etc. These are the programs that the shell runs.
- **Graphical User Interface (GUI):** The visual desktop environment (e.g., GNOME, KDE, XFCE) that provides windows, icons, and menus. It runs on top of the core system.

2. Basic Shell Commands

The shell is a powerful tool. Here are some of the most fundamental commands:

- **ls:** List files and directories.
 - ls -l: List in long format (shows permissions, owner, size, date).
 - ls -a: List all files, including hidden ones (those starting with a dot).
- **cd:** Change directory.
 - cd /home/user/documents: Go to a specific directory.
 - cd ..: Go up one level to the parent directory.
 - cd ~ or cd: Go to your home directory.
- **pwd:** Print working directory (shows you where you currently are).
- **mkdir:** Make directory.
 - mkdir new_folder: Creates a new directory named new_folder.
- **rmdir:** Remove directory (only works on empty directories).
- **touch:** Create an empty file.
 - touch new_file.txt
- **cp:** Copy a file or directory.
 - cp source.txt destination.txt
- **mv:** Move or rename a file or directory.
 - mv old_name.txt new_name.txt (rename)
 - mv file.txt ../another_folder/ (move)

- **rm:** Remove (delete) a file.
 - `rm file_to_delete.txt`
 - `rm -r folder_to_delete:` Recursively remove a directory and all its contents (use with caution!).
- **cat:** Concatenate and display the contents of a file.
 - `cat my_file.txt`
- **man:** Display the **man**ual page for a command.
 - `man ls:` Shows you all the options for the `ls` command. This is your best friend when learning Linux.